



Universidad de Jaén

Tema 2.3 Jakarta EE, Capa de presentación

Jakarta Faces, JSF

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

Este obra está bajo una [Licencia Creative Commons](https://creativecommons.org/licenses/by-sa/4.0/)
Atribución-CompartirIgual 4.0 Internacional.

v 3.4.2

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Objetivos

- Conocer las características de **Jakarta Faces, JSF** como Framework de desarrollo web en la plataforma JEE
- Saber construir vistas con la tecnología de **Facelets** y organizar el contenido usando **plantillas** y **componentes personalizados**
- Saber instanciar y controlar la gestión de objetos en JSF mediante **Inversión de control** e **inyección de dependencias**
- Saber gestionar información de **peticiones GET y POST**
- Conocer y usar el modelo de **navegación web** en JSF
- Controlar **operaciones asíncronas** desde vistas JSF
- Saber **convertir y validar en servidor información** de formularios
- Conocer algunas **bibliotecas de componentes JSF**



Jakarta Faces

- **Módulo de desarrollo de aplicaciones web arquitectura Jakarta EE**

- Versión 4.0 Mayo 2022 Jakarta 10

- **Características**

- **Modelo de componentes UI** con estado asociado en servidor y eventos
- Basado en **MVC (MVVM, pull-based)**: Las URLs se enrutan hacia las vistas
- Separación entre visualización de componentes y comportamiento
- Soporte de **conversión, validación y enlazado (binding)** entre presentación y modelo
- Actualizaciones asíncronas de partes de la vista (tecnología AJAX)



- **Arquitectura**

- Construido sobre arquitectura *Servlets*
- **Controlador principal**: control de flujo de ejecución con **enrutamiento a vistas**
- Uso de **Facelets** (xhtml y *custom tags*) para creación de vistas
- **Beans** para representar el estado (modelo de vista) y la lógica de control de las vistas (controlador)

3

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Modelo de componentes en aplicaciones web

- **Objetivo:**

- Replicar el modelo tradicional de programación GUI en el escritorio basado en componentes reutilizables, eventos que generan y observadores que atienden sus peticiones (e.g. Java awt/swing/fx)

- **Problemas a resolver:**

- HTTP no orientado a conexión (sin estado)
- Diferencias entre programación en lado cliente (html/javascript) y servidor (java)

- **Solución:**

- **Modelar** totalmente en el servidor la estructura jerárquica (componentes) de la vista utilizando clases Java. Los componentes almacenan su estado
- **Generar** automáticamente HTML/javascript
- **Simular el modelo de eventos/manejadores** entre la representación html/javascript de la vista y los elementos equivalentes en el servidor

- **Resultado:**

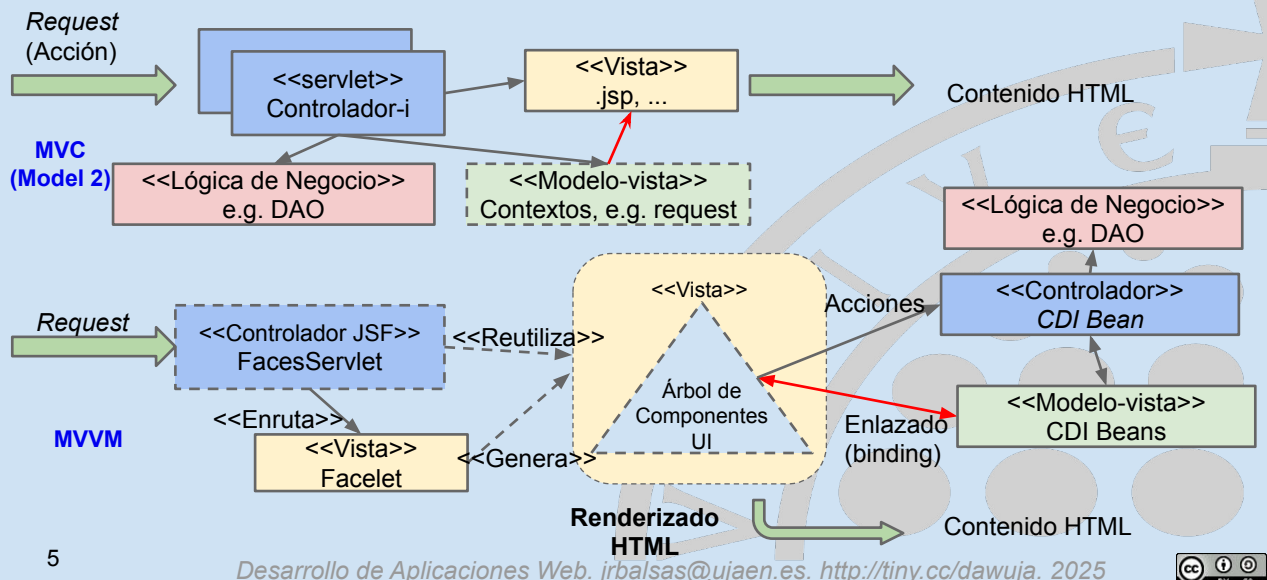
- Programar aplicaciones WEB de forma análoga a aplicaciones de escritorio usando un único lenguaje

4

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

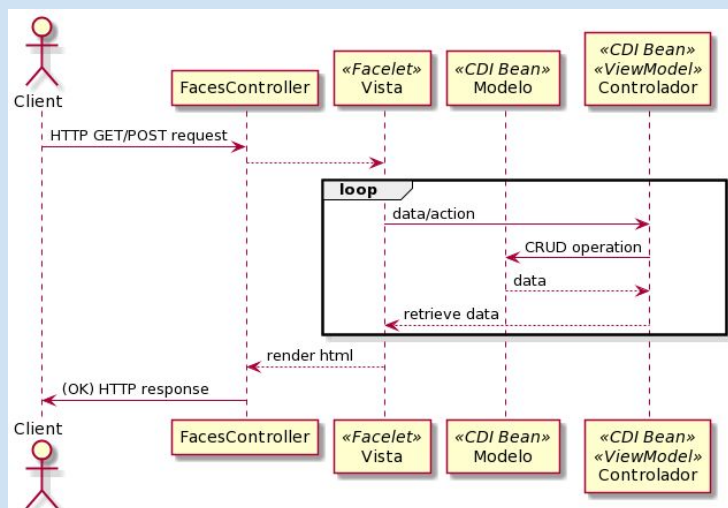


Frameworks basados en acción frente a basados en componentes



5

Diagrama de secuencia Jakarta Faces



El modelo de vista puede ser cualquier bean gestionado (CDI), incluso el propio controlador

6

Ciclo de vida de una petición en JSF

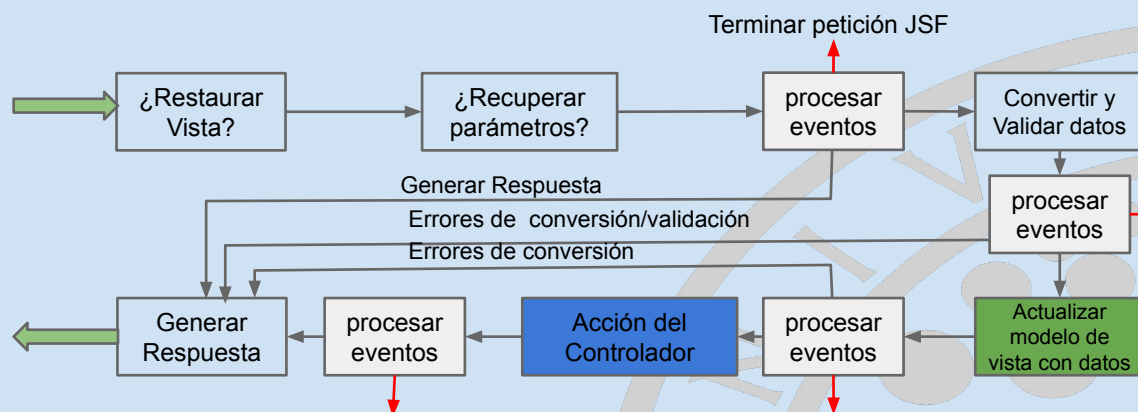
- Establece cómo se procesa una petición y cómo se genera la respuesta
- **Fases:**
 - **Localizar la vista** solicitada
 - Permitir a los componentes **obtener los datos o parámetros** enviados que tengan asociados
 - **Conversión** de datos
 - **Validación** de datos
 - Avisar a cualquier manejador de **eventos** (*listeners*) que sea necesario
 - Actualizar el **modelo de vista**
 - Seleccionar y generar la **nueva vista**

7

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ciclo de vida petición JSF



8

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Vistas en JSF

Facelets y Templates

Facelets

- **Lenguaje de definición de vista basado en etiquetas xml**
 - Ficheros con **extensión .xhtml**
 - usar *minúsculas, etiquetas auto-cerradas*, etc. para garantizar la corrección del documento. Sustituir & por *&*;
 - NO permite código java
- Sirve para **construir el árbol de componentes**. Procesado más **eficiente** que JSP
- Soporte de **Expression Language**
- Uso de **plantillas (templates)** para crear páginas y **componentes**
- Uso de **bibliotecas de etiquetas predefinidas** y personalizadas
 - **Propias JSF**: *facelets (ui:)*, *html (h:)* y *core (f:)*
 - **JSTL**: sólo *core (c:)* y *function (fn:)*
 - Se utilizan espacios de nombres para referenciarlas

```
<!DOCTYPE html>
<html lang="es"
  xmlns:ui="jakarta.faces.facelets"
  xmlns:f="jakarta.faces.core"
  xmlns:h="jakarta.faces.html"
  xmlns:c="jakarta.tags.core">
```

Por compatibilidad también se soportan los identificadores antiguos con prefijo, <http://xmlns.jcp.org/jsf/> e.g. <http://xmlns.jcp.org/jsf/facelets>

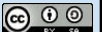
Acceso a Facelets

- Los facelets deben ser procesados por el *controlador principal* de JSF para generar la estructura.
 - Activación automática: clase anotada `@FacesConfig` o fichero `/WEB-INF/faces-config.xml`
- Ubicados** en raíz web (pública) de la aplicación `src/main/webapp`
- Configuración**
 - Por defecto** el controlador frontal se vincula a las siguientes URLS (puede personalizarse en `web.xml`)
`/faces/*, *.jsf, *.faces, *.xhtml`
 - Ejemplos de acceso a fichero `index.xhtml` en raíz
 - `http://localhost:8080/appweb/index.jsf`
 - `http://localhost:8080/appweb/index.xhtml`
 - `http://localhost:8080/appweb/faces/index.xhtml`
 - `http://localhost:8080/appweb/index.faces`
 - Pueden pasarse **parámetros de configuración** al controlador frontal (en `web.xml`)

```
<context-param>
  <!-- Enable verbose error messages -->
  <param-name>javax.faces.PROJECT_STAGE</param-name>
  <!-- Other values: Production, SystemTest, UnitTest -->
  <param-value>Development</param-value>
</context-param>
```

11

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de Facelet

Fichero: `/index.xhtml`

```
<!DOCTYPE html>
<html
  xmlns:h="jakarta.faces.html"
  xmlns:ui="jakarta.faces.facelets">
  <h:head>
    <title>Inicio</title>
  </h:head>
  <h:body>
    <h1>Gestión de Club</h1>
    <h:graphicImage library="images"
      name="logo.png" alt="Logo Club"/>
    <h:link value="Gestión de clientes"
      outcome="/cliente/listado"/>
    <ui:debug/>
  </h:body>
</html>
```

<ui:debug/> permite mostrar en el navegador el árbol de componentes, variables en ámbitos, etc. pulsando **Ctrl+Shift+D**

Árbol de componentes en servidor

```
<UIViewRoot id="j_idt1" inView="true" locale="es_ES" renderKitId="HTML_BASIC"
rendered="true" transient="false" viewId="/test.xhtml">
  <!DOCTYPE html>
  <html xmlns="http://www.w3.org/1999/xhtml">
    <UIOutput id="j_idt3" inView="true" rendered="true" transient="false">
      <title>Inicio</title>
    </UIOutput>
    <UIOutput id="j_idt5" inView="true" rendered="true" transient="false">
      <h1>Gestión de Club</h1>
      <HtmlGraphicImage alt="Logo Club" id="j_idt7" inView="true"
ismap="false" rendered="true" transient="false"/>
      <HtmlOutcomeTargetLink disabled="false" id="j_idt8" inView="true"
includeViewParams="false" outcome="/cliente/listado"
rendered="true" transient="false" value="Gestión de clientes"/>
    </UIOutput>
    <UIDebug hotkey="D" id="j_idt10" inView="true" rendered="true"
transient="true"/>
  </html>
</UIViewRoot>
```

12

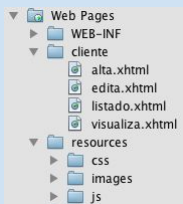
Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de uso CRUD en JSF

http://localhost:8080/club/cliente/inicio.jsf

Fuentes en <https://github.com/jrbalsas/dawClubJSF>



A lo largo del tema utilizaremos fragmentos de esta aplicación para estudiar los conceptos



cliente/borra.jsf?id=11

cliente/alta.jsf

cliente/visualiza.jsf?id=11

cliente/edita.jsf?id=11

Alta Cliente

Nombre:

DNI:

Socio: ☐

13 Guardar Cancelar

Visualiza Cliente

ID: 11

Nombre: Gregorio

DNI: 1234567

Socio: Sí

Volver

Edita Cliente

ID: 11

Nombre:

DNI:

Socio: ☒

Guardar Cancelar

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. http://tiny.cc/dawuja. 2025



Etiquetas básicas JSF

- Selección biblioteca: `<html xmlns:h="jakarta.faces.html">`
- Consulta on-line: <https://jakarta.ee/learn/docs/jakartaee-tutorial/>

Función	Descripción
<code><h:head></h:head></code> <code><h:body></h:body></code>	- Cabecera y contenido de documento HTML. Es conveniente utilizarlas ya que otros componentes las localizan para interactuar con ellas.
<code><h:outputText value="txt"></code> <code><h:outputFormat value="texto {0}"></code>	- Visualización de texto o variable EL. También <code>#{variable}</code> - Texto con 'huecos' parametrizables con etiquetas anidadas <code><f:param value="txt"/></code>
<code><h:link outcome= ></code> <code><h:outputLink value= ></code>	- Creación de un enlace a otra página jsf (outcome="vista"), i.e. <code></code> - Enlace directo a recurso admiten <code><f:param name= value= ></code> anidados para añadirlos a query-string del enlace (y evitar &)
<code><h:graphicImage /></code>	- Visualiza una imagen con etiqueta <code></code>
<code><h:outputStylesheet /></code>	- Enlaza una hoja de estilos, i.e. link
<code><h:outputScript /></code>	- Enlaza un fichero javascript, i.e. script
<code><h:panelGrid ></code> <code><h:panelGroup></code>	- Muestra contenidos organizándolos mediante etiquetas html e.g. <code><table></code> - Agrupa elementos, i.e. div
<code><h:message for=></code> <code><h:messages></code>	- Muestra los mensajes generados por un componente (for=) - Muestra los mensajes de todos los componentes de la vista

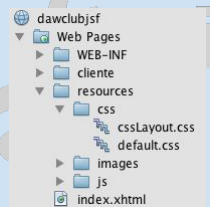


Organización de contenidos en vistas

Recursos, Templates y Composite Components

Uso recursos estáticos en facelets

- **Imágenes, css, javascript**, etc. ubicados en subcarpetas (library) de **directorio resources** de parte pública web
- Selección mediante atributos de etiquetas específicas.
- **Ventaja:** independiente de url del facelet
- **Tipos:**
 - `<h:graphicImage library="images" name="file1.jpg"/>`
 - `<h:outputStylesheet library="css" name="layout.css"/>`
 - `<h:outputScript library="js" name="main.js target="head"/>`
 - También se puede obtener la url del recurso con `#{resource[library:name]}`
- Si no se especifica el atributo library, las rutas absolutas se refieren a la raíz de la appweb (**ContextPath**)
 - e.g. `<h:graphicImage value="/logo.jpg"/>` `//→ /contextpath/logo.jpg`
 - `<h:outputStylesheet value="/bootstrap/bootstrap-min.css"/>`



Creación de Facelets Templates

- **Definición:** un facelet "**template**" es un *facelet con zonas* que pueden sustituirse por elementos definidos en un facelet "cliente"
- Un **facelet "cliente"** hace referencia a la plantilla que utiliza y sólo define los elementos de ésta que sustituirá. No es un documento html completo (aunque sí válido)
- Es el mecanismo fundamental para crear nuevos componentes
- Etiquetas habituales: `<html xmlns:ui="jakarta.tags.facelets">`

Función	Descripción
<code><ui:insert name="elemento"></code> Contenido por defecto <code></ui:insert></code>	Declara elemento como zona a sustituir en template . Si no se proporciona por el facelet cliente se sustituye por el contenido por defecto
<code><ui:composition template="plantilla.xhtml"></code> <code><!-- contenidos template --></code> <code></ui:composition></code>	Define un facelet cliente que usará una plantilla determinada. También sirve para definir "fragmentos" a usar con <code><ui:include></code>
<code><ui:define name="elemento"></code> contenido <code></ui:define></code>	En un facelet cliente , define el contenido que sustituirá a una zona en la plantilla
<code><ui:include src="fichero.xhtml"/></code>	Se reemplaza por el contenido de otro fichero

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de plantilla y cliente

Fichero: /WEB-INF/template/plantilla.xhtml

1

```

<!DOCTYPE html>
<html
  xmlns:h="jakarta.faces.html"
  xmlns:ui="jakarta.faces.facelets">
</h:head>
  <h:outputStylesheet library="css" name="layout.css"/>
  <h:outputScript library="js/jquery" name="jquery.js"/>
</h:head>
<h:body>
<header>
  <ui:insert name="cabecera">
    <ui:include src="/WEB-INF/inc/header.xhtml"/>
  </ui:insert></header>
<main>
  <nav>
    <p>Menú</p>
    <ui:insert name="menu">
    </ui:insert>
  </nav>
  <section>
    <ui:insert name="contenido">
    </ui:insert>
  </section>
</main>
<footer>
  <ui:include src="/WEB-INF/inc/footer.xhtml"/>
</footer>
</h:body>
</html>
  
```



Fichero: /inicio.xhtml

```

<!DOCTYPE html>
<html
  xmlns:h="jakarta.faces.html"
  xmlns:ui="jakarta.faces.facelets">
  <ui:composition
    template="/WEB-INF/template/plantilla.xhtml">
    <ui:define name="menu">
      <h:link value="Gestión de clientes"
        outcome="/cliente/listado"/>
    </ui:define>
    <ui:define name="contenido">
      <h1>Gestión de Club</h1>
      <h:graphicImage library="images"
        name="logo.png" alt="Logo Club"/>
    </ui:define>
  </ui:composition>
</html>
  
```

2

3

Importante: lo que queda fuera de etiquetas `<ui:define>` no aparecerá en la vista resultante

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Composite Components

- Para favorecer la reutilización, se pueden crear **componentes nuevos** a partir de otros existentes, incluso dotándolos de comportamiento

• Creación:

- Definidos en subcarpeta de **resources** de contenidos públicos
- Fichero .xhtml con definición atributos , e.g. /resources/**daw/cliente.xhtml**

```
<!DOCTYPE html >
<html xmlns:cc="jakarta.faces.composite">
  <cc:interface>
    <cc:attribute name="value" class="com.daw.club.model.Cliente" required/>
  </cc:interface>

  <cc:implementation>
    <ul class="panel-body list-group">
      <li class="list-group-item"><strong>ID:</strong> #{cc.attrs.value.id}</li>
      <li class="list-group-item"><strong>Nombre:</strong> #{cc.attrs.value.nombre}</li>
      <li class="list-group-item"><strong>DNI:</strong> #{cc.attrs.value.dni}</li>
      <li class="list-group-item"><strong>Socio:</strong> #{cc.attrs.value.socio?"Sí":"No"}</li>
    </ul>
  </cc:implementation>
</html>
```

• Uso:

- Vincular espacio de nombres a carpeta de componentes

```
<html ... xmlns:daw="jakarta.faces.composite/daw">
...
<daw:cliente value="#{clienteCtrl.cliente}" />
```

Controladores en JSF

Managed Beans

Gestión de objetos en JakartaEE

- **Problema**

- ¿Cómo controlar eficazmente el **ciclo de vida y las relaciones entre objetos** en aplicaciones JEE?

- **Inversión de control (CI)**

- La aplicación **delega en el servidor** de aplicaciones o *framework*:
 - la **instanciación y destrucción** de objetos
 - el establecimiento de **relaciones** entre objetos

- **Inyección de dependencias (DI)**

- **Los objetos** no crean o buscan los objetos con los que se relacionan; **declaran qué dependencias tienen**
- El *framework* se encarga de crear los objetos y establecer relaciones entre clases asignando (*inyectando*) valores a sus atributos
- Cambios en relaciones sin necesidad de modificar código o recompilar. Utilizar *interfaces* para permitirlo

21

Instanciación de componentes JEE

- **Context and Dependency Injection, CDI 4.0**

- Disponible desde JEE>=6
- Basada principalmente en **anotaciones**
- Fichero `/WEB-INF/beans.xml` para activar la búsqueda de componentes a instanciar por el contenedor (*Managed Beans*). JEE7+ búsqueda por anotaciones.

- **CDI managed beans**

- Objetos creados directamente por el contenedor cuando son necesarios
- Su **persistencia** se indica con anotaciones:
 - `@ApplicationScoped`
 - `@SessionScoped`
 - `@RequestScoped`
 - `@Dependent`, la del bean donde se inyecte
 - `@ViewScoped`, mientras no se cambie de vista, e.g. formularios interactivos, ajax
(ojo, en paquete `jakarta.faces.view.ViewScoped`)
 - `@ClientWindowScoped`, mientras se conserve el valor de un parámetro `jfwid` en una misma ventana
 - `@FlowScoped`, durante varias vistas, e.g. proceso de pago

en paquete `jakarta.enterprise.context`

22

Modelo para pruebas

@ApplicationScoped

public class **ClienteDAO** implements Serializable {

```
private Map<Integer, Cliente> clientes;
private int idCliente = 1; //last clientID

public ClienteDAO() {
    clientes = new HashMap<>();
};
public List<Cliente> buscaTodos() {
    return clientes.values().stream().collect( Collectors.toList() );
}
public Cliente buscald(int id) {
    return clientes.get(id)
}
public boolean crea(Cliente c) {
    c.setId( idCliente++ ); //registered clients have sequential IDs
    clientes.put(c.getId(), new Cliente(c) );
    return true;
}
```

- DAO, *Data Access Object*, abstracción para el acceso a datos (se estudiará más adelante)
- Simula una colección de datos, almacenados en memoria mediante un HashMap y compartidos por todos los usuarios

```
public boolean guarda(Cliente c) {
    boolean result=false;
    if ( clientes.containsKey( c.getId() ) ) {
        Cliente nc=new Cliente(c);
        clientes.replace( c.getId(), nc );
        result=true;
    }
    return result;
}
public boolean borra(int id) {
    boolean result=false;
    if ( clientes.containsKey(id) ) {
        clientes.remove(id);
        result = true;
    }
    return result;
}
```

23

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Controlador y Modelo de vista JSF

• Controlador en JSF

- Clases (beans) gestionadas por el Servidor mediante **Inversión de control**:
 - anotaciones habituales CDI, e.g. @RequestScoped o @ViewScoped
- Se les asigna un nombre con @Named para localizarse desde vistas con EL:
@Named(value = "clienteCtrl")
@ViewScoped
public class ClienteController implements Serializable {
 - La vista puede invocar métodos: #{ clienteCtrl.método(param) }

• Modelo de vista

- Son beans asociados a componentes de la vista. Almacenan su **estado**
 - Sintaxis diferente con EL: #{managedBean.propiedad}
 - Lectura y "modificación" de propiedades (*data-binding*)
- Pueden ser una propiedad de un controlador o un bean independiente
 - e.g. #{clientesCtrl.cliente.nombre} //i.e. clientesCtrl.getCliente().getNombre()
- Las vistas también acceden a los **objetos implícitos EL**: param, request, session, header, cookie, ... y dos nuevos: view y facesContext

24

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Inyección de dependencias en JEE

- Problema

- Como la persistencia del modelo es distinta a la del controlador ¿cómo puede acceder el controlador al modelo?

- Inyección de dependencias

- Anotación `@Inject` asociada a propiedad (constructores o métodos set) de bean
- El servidor localiza beans compatibles, los instancia, si es necesario, e inyecta en el atributo

- Ejemplo

```
@ApplicationScoped
public class ClienteDAO implements Serializable {
    // ...
    public boolean crea(Cliente c) {...}
    public List<Cliente> buscaTodos() {
        return clientes.values().stream.collect(Collectors.toList());
    }
}
```

```
@ApplicationScoped
public class AppConfig {
    @Inject private ClienteDAO clientes; // model
    public void onStartUp( @Observes Startup event ) {
        // Create sample customers in DAO on application start-up
        clientes.crea( new Cliente( 0, "Paco López", "11111111A", false ) );
        clientes.crea( new Cliente( 0, "María Jiménez", "22222222B", true ) );
    }
}
```

```
@Named(value=clienteCtrl)
@ViewScoped
public class ClienteController implements Serializable {
    @Inject
    private ClienteDAO clientes; // model

    private Collection<Cliente> lc; // view-model

    public ClienteController() {
        //DAO not injected yet
    }
    @PostConstruct
    public void init() { //DAO injected!
        lc=clientes.buscaTodos();
    }
}
```

25

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Iteración sobre elementos

- Importante:

- Las etiquetas JSTL, e.g. `c:forEach`, solo se procesan al generar el árbol de la vista, pero NO en las sucesivas iteraciones entre cliente y servidor (e.g. cambio en valores de propiedades)

- Iteración sobre colecciones de tipo Array y List

```
<ui:repeat var="variable" value="#{bean.colección}" varStatus="vStatus" offset="0" size="10" step="1" >
    #{variable...} <!-- más control sobre el código de cada línea -->
</ui:repeat>
```

- Visualización de tablas de elementos (Array y List)

```
<h:dataTable var="variable" value="#{bean.colección}"
    styleClass="clase1" headerClass="clase2" rowClasses="clase3 clase4">
    <h:column>
        <f:facet name="header" >Título columna1</f:facet>
        #{variable.prop1}
    </h:column>
    <h:column>
        <f:facet name="header" >Título columna2</f:facet>
        #{variable.prop2}
    </h:column>
    <!-- más columnas -->
</h:dataTable>
```

26

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Listar Clientes

`<ui:composition ... template="/WEB-INF/plantilla/entidad.xhtml">` `/cliente/listado.jsf`

```
<ui:define name="content">
  <h1>Listado de Clientes </h1>
  <h:form>
    <h:dataTable var="cliente" value="#{clienteCtrl.clientes}">
      <h:column> <f:facet name="header">ID</f:facet>
        #{cliente.id} </h:column>
      <h:column> <f:facet name="header">Nombre</f:facet>
        #{cliente.nombre} </h:column>
      <h:column> <f:facet name="header">DNI</f:facet>
        #{cliente.dni} </h:column>
      <h:column> <f:facet name="header">Socio</f:facet>
        #{cliente.socio?"SI":"No"} </h:column>
      <h:column> <f:facet name="header">Opciones</f:facet>
        <h:button value="Visualiza" outcome="visualiza" <!-- link to visualiza.jsf?id=1 -->
          <f:param name="id" value="#{cliente.id}" /></h:button> &#160;
        <h:button value="Edita" outcome="edita" <!-- link to edita.jsf?id=1 -->
          <f:param name="id" value="#{cliente.id}" /></h:button> &#160;
        <h:commandButton value="Borra" action="#{clienteCtrl.borra(cliente)}" />
      </h:column>
    </h:dataTable>
  </h:form>
</ui:define>
```

```
@Named(name="clienteCtrl")
@ViewScoped
public class ClienteController
  implements Serializable {

  @Inject ClienteDAO clienteDAO;

  public List<Cliente> getClientes() {
    return clienteDAO.buscaTodos();
  }

  public void borra ( Cliente c ) {
    clienteDAO.borra ( c.getId() );
  }
}
```

Gestión Club					
Menú Inicio Nuevo Cliente	Listado de Clientes				
	ID	Nombre	DNI	Socio	Opciones
	11	Gregorio	1234567	Si	Visualiza Edita Borra
	14	Manuel	1345678A	No	Visualiza Edita Borra
	15	Pedro	5555555-F	No	Visualiza Edita Borra

Condiciones de Uso

```
<ui:define name="left">
  <h:link value="Nuevo Cliente" outcome="alta"/><br/>
</ui:define>
</ui:composition>
```

27

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Recepción de parámetros GET

- **Peticiones GET con parámetros**
 - e.g. <http://host/app/cliente/visualiza.jsf?id=45>
- **Problema**
 - Como la vista recibe la petición, se genera antes de que el controlador pueda obtener datos concretos
 - Es posible que la vista necesite algunos datos dependiendo de algún parámetro GET
- **Solución**
 - `<f:viewParam name= value= />` recupera un parámetro (*name*) y lo pasa a una variable del modelo (*value*). Se pueden utilizar varias entradas
 - `<f:viewAction action= />` invoca al controlador (*action*) para que modifique el modelo de vista antes de que se visualice la vista
 - e.g. para llamar al DAO con datos recuperados de parámetros
 - Ambas se utilizan dentro de la etiqueta `<f:metadata>`

28

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Visualiza Cliente

/cliente/visualiza.jsf?id=45

<ui:composition ...>

<ui:define name="content">

<f:metadata>

<f:viewParam name="id"
value="#{clienteCtrl.cliente.id}"
required="true" />

<f:viewAction action="#{clienteCtrl.recupera}" />

</f:metadata>

<h:messages class="text-danger"/>

<ui:fragment rendered="#{clienteCtrl.cliente != null}">

<h1>Visualiza Cliente</h1>

ID: #{clienteCtrl.cliente.id}

Nombre: #{clienteCtrl.cliente.nombre}

DNI: #{clienteCtrl.cliente.dni}

Socio: #{clienteCtrl.cliente.socio?"SI":"No"}

<h:button value="Editar" outcome="edita">

<f:param name="id" value="#{clienteCtrl.cliente.id}" />

</h:button>

<h:commandButton value="Borrar"

action="#{clienteCtrl.borra}" />

</ui:fragment>

</ui:define>

</ui:composition>

1 llamada método
c.setId(valor)

@Named(name="clienteCtrl")

@ViewScoped

public class ClienteController

implements Serializable {

private Cliente c = new Cliente(); //view-model

@inject

FacesContext fc;

public Cliente getCiente () { return c; }

/** preload view-model: client id obtained from param:*/

public void recupera() {

this.c=clienteDAO.buscald(c.getId());

if (c ==null) {

fc.addMessage(null, new FacesMessage

("El cliente indicado no existe"));

}

}

Visualiza Cliente

ID: 45

Nombre: Gregorio

DNI: 1122112-R

Socio: No

Editar

Borrar

Volver

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. http://tiny.cc/dawuja. 2025



Navegación web

● Implícita:

- los **componentes que generan acciones** (e.g. botón) pueden saltar a una vista cuyo nombre se indica en atributos específicos
- nombre de vista** (sin .xhtml) establecido en el valor del atributo
 - <h:link value="Nuevo Cliente" outcome="alta"/>
 - <h:commandButton action="inicio" value="Cancelar"/>
- valor de atributo como **resultado (cadena) de un método** del controlador
 - <h:commandButton action="#{cliente.guardar}" value="Guardar"/>
 - Una **acción vacía ""** equivale a quedarse en la vista actual (e.g. hay errores)
 - Por defecto la **navegación en acciones es de tipo forward** (no cambia la URL). Para forzar una **redirección** añadir **?faces-redirect=true** a acciones implícitas o etiqueta especial en reglas explícitas.
 - return "cliente/visualiza?faces-redirect=true&id="+cliente.getId();

● Explícita

- Se modela mediante un **grafo dirigido** formado por vistas (nodos) y acciones (aristas dirigidas) en fichero **faces-config.xml** mediante reglas del tipo (**vistaorigen, acción, vistadestino**). ¡Herramientas gráficas!



Navegación: Visualiza/Borra Cliente

/cliente/visualiza.jsf?id=45

```
<ui:composition ...>
<ui:define name="content">
<f:metadata>
<f:viewParam name="id"
value="#{clienteCtrl.cliente.id}"
required="true" />
<f:viewAction action="#{clienteCtrl.recupera}" />
</f:metadata>
<h:messages class="text-danger"/>
<ui:fragment rendered="#{clienteCtrl.cliente != null}">
<h1>Visualiza Cliente</h1>
ID: #{clienteCtrl.cliente.id}<br/>
Nombre: #{clienteCtrl.cliente.nombre}<br/>
DNI: #{clienteCtrl.cliente.dni}<br/>
Socio: #{clienteCtrl.cliente.socio?"S":"No"}<br/>
<h:button value="Editar" outcome="edita">
<f:param name="id" value="#{clienteCtrl.cliente.id}"/>
</h:button> <!-- static link to edita.jsf?id=1 -->
<h:commandButton value="Borrar"
action="#{clienteCtrl.borra}"/>
</ui:fragment>
</ui:define>
<ui:define name="left">
<!-- static link to listado.jsf -->
<h:link value="Volver" outcome="listado"/><br/>
</ui:define>
</ui:composition>
```

```
@Named(name="clienteCtrl")
@ViewScoped
public class ClienteController
implements Serializable {
private Cliente c = new Cliente();

public Cliente getCliete () { return c; }

public String borra() {
// Delete current view-model client
clienteDAO.borra( this.c.getId() );
//OP1 (forward), url remains /cliente/visualiza.jsf
return "listado";
//OP2 (redirect) url changes to /cliente/listado.jsf
return "listado?faces-redirect=true";
}
}
```

Visualiza Cliente
ID: 45
Nombre: Gregorio
DNI: 1122112-R
Socio: No

31

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Gestión de formularios

Etiquetas y actualizaciones asíncronas



Gestión de formularios: etiquetas

- Selección biblioteca: `<html xmlns:h="jakarta.core.html">`
- Consulta on-line: <https://jakarta.ee/learn#documentation>

Función	Descripción
<code><h:form></h:form></code>	Definición de formulario (¡sin action!)
<code><h:inputText value=/"></code> <code><h:inputSecret value=/"></code> <code><h:inputTextarea value=/"></code> <code><h:inputHidden value=/"></code>	Utilizan atributo value para representar el contenido - texto normal - texto oculto (****) - <textarea> - campo oculto
<code><h:commandButton action=/"></code> <code><h:commandLink action=/"></code> <code><h:button></code>	- Botón "submit" del formulario. Invoca, a una acción en controlador - Enlace con capacidad de envío, POST, del formulario - Equivalente a un botón en html (para asociar código javascript)
<code><h:selectBooleanCheckbox></code> <code><h:selectOneRadio></code> <code><h:selectManyCheckbox></code>	- Checkbox simple (true/false) - Selección de una opción entre un conjunto - Grupo de checkboxes. - Opciones con etiquetas <code><f:selectedItem></code> o <code><f:selectedItems></code>
<code><h:selectOneMenu></code>	- Lista desplegable - Uso de <code><f:selectedItem></code> o <code><f:selectedItems></code> para opciones
<code><h:selectOneListbox></code> <code><h:selectManyListbox></code>	- Lista de selección única - Lista de selección múltiple - Uso de <code><f:selectedItem></code> o <code><f:selectedItems></code> para opciones

33

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo controlador (I)

```

<!DOCTYPE html>
<html xmlns:h="jakarta.core.html">
<h:head>
  <title>Alta Cliente</title>
</h:head>
<h:body>
  <h1>Alta cliente</h1>
  <h:form>
    Nombre:
    <h:inputText label="Nombre" id="idNombre"
      value="#{clienteCtrl.cliente.nombre}" //clienteCtrl.getClient().get/setNombre()
      required="true"
      requiredMessage="Introduzca un nombre entre 4 y 25 caracteres">
    </h:inputText>
    <h:message for="idNombre" /><br/>
    DNI: <h:inputText label="DNI" id="idDNI"
      value="#{clienteCtrl.cliente.dni}"
      required="true"
      requiredMessage="El DNI es obligatorio">
    </h:inputText>
    <h:message for="idDNI" /><br/>
    Socio: <h:selectBooleanCheckbox label="Socio"
      value="#{clienteCtrl.cliente.socio}" /><br/>
    <h:commandButton value="Guardar" action="#{clienteCtrl.crea}" />
    <h:button value="Cancelar" outcome="index" />
  </h:form>
</h:body>
</html>

```

```

@Named(value="clienteCtrl")
@ViewScoped
public class ClienteController
  implements Serializable{
  // view-model (beans)
  private Cliente c = new Cliente();

  public Cliente getClient() {return c;}

  public String crea() {
    //controller logic (valid bean)
    if (other customized errors)
      return ""; //refresh view with errors
    else
      return "listado?faces-redirect=true";
  }
}

```

Alta Cliente

Nombre:

Introduzca un nombre entre 4 y 25 caracteres

DNI:

El DNI es obligatorio

34

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de controlador: Alta (II)

/cliente/alta.xhtml

```
<ui:composition
  xmlns:h="jakarta.faces.html"
  xmlns:f="jakarta.faces.core"
  xmlns:ui="jakarta.faces.facelets"
  template="/WEB-INF/plantilla/entidad.xhtml">
  <ui:define name="content">
    <h1>Alta Cliente</h1>
    <h:form >
      Nombre: <h:inputText label="Nombre" id="idNombre"
        value="#{clienteCtrl.cliente.nombre}"
        required="true">
    </h:inputText>
    <h:message for="idNombre" /><br/>
    DNI: <h:inputText label="DNI" id="idDNI"
      value="#{clienteCtrl.cliente.dni}"
      required="true">
    </h:inputText>
    <h:message for="idDNI"/><br/>
    Socio: <h:selectBooleanCheckbox label="Socio"
      value="#{clienteCtrl.cliente.socio}" /><br/>
    <h:commandButton value="Guardar"
      action="#{clienteCtrl.crea}" />
    <h:button value="Cancelar" outcome="listado" />
    </h:form>
  </ui:define>
</ui:composition>
```

package daw.club.controller;

```
@Named(value="clienteCtrl")
@ViewScoped
public class ClienteController
  implements Serializable {
```

```
//Business logic
@Inject @DAOJpa
private ClienteDAO clienteDAO;
//View-Model
private Cliente c = new Cliente();
```

```
//View-Model access
public Cliente getCliente() {
  return c;
}
```

```
/** Action: Create a new Client from model data */
public String crea() {
  String view="visualiza?faces-redirect=true&id=";
  c.setId(0);
  clienteDAO.crea(c);
  return view+c.getId(); //Post-Redirect-Get
}
```

1

2.1

2

2.2

3

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Conversión de datos

• Motivación

- En ocasiones es necesario adaptar la información enviada o recibida en la vista
- Ejemplo:** *LocalDate* $\longleftrightarrow$ 9 de enero de 2007, 9/1/2007... o control input

• Mecanismos:

- Implícito:** para acceso con EL a tipos básicos, e.g. Integer
- Declarativo**
 - Atributo **converter** en etiquetas JSF, e.g. `<h:outputText converter=""...>`
 - Etiquetas de conversión especializadas** anidadas a otras etiquetas, e.g. `outputText` o `inputText`. Disponen de parámetros adicionales para personalización, [+info](#)



```
<h:outputText value="#{ctrl.cliente.fechaNacimiento}"
  <f:convertDateTime type="localDate" dateStyle="long" />
</h:outputText>
```

```
<h:outputText value="#{ctrl.articulo.precio}"
  <f:convertNumber maxFractionDigits="3" />
</h:outputText>
```

```
<html xmlns:pt="jakarta.faces.passthrough"
  <!-- Use browser date picker format -->
  <h:inputText pt:type="date"
    value="#{ctrl.cliente.fechaNacimiento}"
    <f:convertDateTime type="localDate"
      pattern="yyyy-MM-dd" />
  </h:inputText>
```

• Conversores personalizados

- Clases desarrolladas por el usuario para convertir tipos de datos específicos, e.g. *TarjetaCrédito* [+info](#)



Ejemplo de controlador: Edición (III)

Acceso: /cliente/edita.jsf?id=11

```
<ui:composition ...
  template="/WEB-INF/plantilla/entidad.xhtml">
  <ui:define name="content">
  <f:metadata>
    <f:viewParam name="id"
      value="#{clienteCtrl.cliente.id}"
      required="true" />
    <f:viewAction action="#{clienteCtrl.recupera()}" />
  </f:metadata>
  <h1>Edita Cliente</h1>
  <h:form>
    ID: #{clienteCtrl.cliente.id}<br/>
    <h:inputHidden value="#{clienteCtrl.cliente.id}" />
    Nombre: <h:inputText label="Nombre" id="idNombre"
      value="#{clienteCtrl.cliente.nombre}"
      required="true">
    </h:inputText>
    <h:message for="idNombre" /><br/>
    DNI: <h:inputText label="DNI" id="idDNI"
      value="#{clienteCtrl.cliente.dni}" required="true">
    </h:inputText>
    <h:message for="idDNI" /><br/>
    Socio: <h:selectBooleanCheckbox label="Socio"
      value="#{clienteCtrl.cliente.socio}" /><br/>
    <h:commandButton value="Guardar" action="#{clienteCtrl.guarda}" />
    <h:commandButton value="Cancelar" action="listado" immediate="true" />
  </h:form>
  </ui:define>
</ui:composition>
```

@Named(value="clienteCtrl")

@ViewScoped

public class ClienteController implements Serializable {

@Inject @DAOJpa
private ClienteDAO clienteDAO;
//View-Model
private Cliente c = new Cliente();

//Model-view access
public Cliente getCliente () { return c; }

/** Get client from id param */
public void recupera() {
 this.c=clienteDAO.buscald(c.getId());
 if (c ==null) {
 fc.addMessage(null, new FacesMessage
 ("El cliente indicado no existe"));
 }
}

public String guarda() {
 clienteDAO.guarda(c);
 return "visualiza"
 + "?faces-redirect=true"
 + "&id="+c.getId();
}

Importante, para cancelar debe prevenirse la fase
de envío (POST) y validación con atributo **immediate**

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Control de opciones de selección

- Disponible para etiquetas de tipo:
 - Radio, Grupos de checkboxes y Select
- Formas para crear las opciones:
 - `<f:selectItem itemValue="val1" itemLabel="Opción 1"/>`
 - `<f:selectItems value="#{bean.cjtoOpciones}"/>`
 - **Array** o **List** de valores
 - **Map** (LinkedHashMap), la **clave** es el texto mostrado y el contenido el **valor** seleccionado. Si el contenido es un objeto, se deben usar los atributos *itemValue* e *itemLabel* para asociarlos a propiedades de dicho objeto. El objeto puede contener datos adicionales (e.g. una imagen)

Ejemplos:

Es socio: `<h:selectBooleanCheckbox value=#{cliente.socio} />`

Método de pago:

```
<h:selectOneRadio value=#{cliente.metodoPago}>
  <f:selectItem itemValue="1" itemLabel="Recibo"/>
  <f:selectItem itemValue="2" itemLabel="Trasferencia"/>
</h:selectOneRadio>
```



Selección múltiple de opciones

Actividades contratadas:

```
<h:selectManyCheckbox value="#{actividadesCliente.actividades}">
  <f:selectItems value="#{actividadesCliente.listaActividades}" />
</h:selectManyCheckbox>
```

@Named //sin nombre equivale a clase en minúscula

@RequestScope

```
class ActividadesCliente {
    private String[] actividades;

    public String[] getActividades() {
        return actividades;
    }
    public void setActividades(String[] nactiv) {
        actividades=nactiv;
    }
    public String[] getListActividades () {
        return new String[]{"Natación", "Tenis", "Petanca"};
    }
}
```

Método de pago:

☒ Recibo ☐ Tráserencia

Actividades contratadas:

☐ Natación ☒ Tenis ☒ Petanca

En actividades se
almacenarán las cadenas
correspondientes a las
opciones seleccionadas

39

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Actualizaciones asíncronas en JSF

- Los componentes (e.g. botón) de un formulario pueden adaptarse para lanzar **peticiones http asíncronas** (AJAX) y **actualizar solo partes de la página**
- La petición enviará ciertos datos del formulario, ejecutará una acción y actualizará sólo ciertos componentes
- **Formato general:**

```
<h:commandButton ... action=" ..." >
  <f:ajax execute="id1 id2" render="id3 id4"
    event="evdom" onevent="javascriptAjaxFunc" />
</h:commandButton>
```

• Elementos (opcionales):

- **execute**, id de elementos a enviar al servidor o @this, @form, @none, @all
- **render**, id de elementos a recargar del servidor
- **event**, evento DOM que activa la petición. e.g. blur, click, etc. Dependen del control. Cada control tiene uno por defecto. *valueChange*, evento "simulado" para unificar el comportamiento de cambios de valor en controles
- **onevent**, función javascript para atender cambios en petición asíncrona
- La acción del botón debe devolver "" (cadena vacía) para permanecer en la página
- El controlador debe tener al menos ámbito **@ViewScoped**

40

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Actualizaciones asíncronas en JSF

- Los componentes (e.g. botón) de un formulario pueden adaptarse para lanzar **peticiones http asíncronas** (AJAX) y **actualizar solo partes de la página**
- La petición enviará ciertos datos del formulario, ejecutará una acción y actualizará sólo ciertos componentes
- **Formato general:**

```
<h:commandButton ... action=" ..." >
    <f:ajax      execute="id1 id2" render="id3 id4"
                event="evdom" onevent="javascriptAjaxFunc" />
</h:commandButton>
```

- **Elementos (opcionales):**

- **execute**, id de elementos a enviar al servidor o @this, @form, @none, @all
- **render**, id de elementos a recargar del servidor
- **event**, evento DOM que activa la petición. e.g. blur, click, etc. Dependen del control. Cada control tiene uno por defecto. *valueChange*, evento "simulado" para unificar el comportamiento de cambios de valor en controles
- **onevent**, función javascript para atender cambios en petición asíncrona
- La acción del botón debe devolver "" (cadena vacía) para permanecer en la página
- El controlador debe tener al menos ámbito **@ViewScoped**

41

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo Actualización Asíncrona

- Mostrar una lista de selección si se activa un checkbox:

Alta Cliente

Nombre: Desconocido

DNI:

Socio: ☐

Nombre: Desconocido

DNI:

Socio: ☒

Tipo Socio:

- Gold
- Premium
- Standard

```
<h:form>
  <!-- ... -->
  Socio:<h:selectBooleanCheckbox label="Socio" value="#{clienteCtrl.cliente.socio}"
    <f:ajax render="idTipoSocio" execute="@this"/>
  </h:selectBooleanCheckbox><br/>
  <h:panelGroup id="idTipoSocio" >
    <h:panelGrid columns="2" rendered="#{clienteCtrl.cliente.socio}">
      Tipo Socio:
      <h:selectOneListbox required="true" value="#{clienteCtrl.cliente.tipoSocio}">
        <f:selectItem itemValue="1" itemLabel="Gold" />
        <f:selectItem itemValue="2" itemLabel="Premium" />
        <f:selectItem itemValue="3" itemLabel="Standard" />
      </h:selectOneListbox>
    </h:panelGrid>
  </h:panelGroup>
  <h:commandButton value="Guardar" action="#{clienteCtrl.crea}" />
  <h:commandButton value="Cancelar" action="index" immediate="true" />
</h:form>
```

Al marcar, enviar solo este valor (@this) y actualizar el componente con id indicado

El html se obtiene al marcar el checkbox

Pasar a vista sin procesar datos (post) ni perder estado (e.g. con redirect)

42

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Validación de datos en servidor con JSF

Validación de datos

- ¿Cuándo?
 - Después de convertir el dato al tipo esperado
 - Antes de asignar el dato a la propiedad del bean
- Tipos:
 - **Estándar JEE:** Anotaciones *Bean Validation* (tema 2.4). Validación en modelo. **Recomendada**
 - **Declarativa JSF**
 - Validación en vistas
 - atributo **required** (=true"). Obliga a que se introduzca un dato.
 - **Reglas explícitas** con etiquetas anidadas específicas:
 - `<f:validateLength minimum="v1" maximum="v2"/>`
 - `<f:validateLongRange minimum="v1" maximum="v2"/>`
 - `<f:validateDoubleRange minimum="v1" maximum="v2"/>`
 - `<f:validateRegex pattern="expReg"/>`
 - **Método específico** del controlador llamado desde atributo **validator** en componente
 - Validación en controlador
 - El método recibe tres parámetros:
 - *FacesContext*, normalmente para envío de mensajes
 - *UIComponent*, componente que se está validando
 - *Object*, elemento enviado a validar

Visualización de mensajes

- JEE dispone de **mensajes de error predeterminados** para cada violación de valor o conversión.

- Mensajes personalizados**

- En atributos de controles del formulario**

- Restricción: *required* → *requiredMessage*
 - Reglas explícitas, e.g. `<f:validateLength>` → *validatorMessage*
 - Errores de conversión → *converterMessage*

- Mensajes/errores generados desde controlador o incluso el modelo**

- Asignados a reglas *Bean Validation* (tema 2.4)
 - Objetos ***FacesMessages*** asociados a componente o sin asociar

```
FacesMessage msg=new FacesMessage("El dni está duplicado")
fc.addMessage( idComponente , msg ); // @Inject FaceContext fc
fc.addMessage( "", new FacesMessage( // Error genérico no asociado a un componente en particular
    FacesMessage.SEVERITY_ERROR, "El cliente no existe", "No existe un cliente con ese id"
));
```

Summary Detail

- Visualización de mensajes generados en la vista:**

- `<h:message for="idComponente" class="estilo" showDetail="true/false" showSummary="false/true">`
 - Muestra los errores asociados a un componente
 - `<h:messages class="estilo" errorClass="text-danger" infoClass="text-info">`
 - Muestra todos los errores del formulario y errores sin asociar

45

 Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025


Ejemplos de Validación JSF

```
<h:panelGrid columns="3">
  Edad: <h:inputText id="idEdad" value="#{usuario.edad}" required="true"
    converterMessage="Debe ser un número"
    <f:validateLongRange minimum="16" maximum="100"/>
  </h:inputText>
  <h:message for="idEdad"/>
  DNI: <h:inputText id="idDni" value="#{usuario.dni}" required="true"
    validator="#{usuarioCtrl.validarDNI}"/>
  <h:message for="idDni"/>
  E-mail: <h:inputText id="idMail" value="#{usuario.mail}"
    validatorMessage="Correo no válido"
    <f:validateRegex pattern="^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,4}$"/>
  </h:inputText>
  <h:message for="idMail"/>
</h:panelGrid>
```

Edad:	
DNI:	
E-mail:	

```
@Named @ViewScoped
class UsuarioCtrl{
  //...
  public void validarDNI (FacesContext fcontext, UIComponent inputDni, Object value) {
    String dni=(String)value;
    if ( !dni.matches("[0-9]{7,8}-?[a-zA-Z]{1,2}") ) {
      ((UIInput) inputDni).setValid(false); //discard input data from view
      FacesMessage msg = new FacesMessage("Formato DNI no válido: 12345678-A");
      fcontext.addMessage( "idDNI" , msg); //error message
    }
  }
}
```

46

 Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025


Bibliotecas de utilidad/componentes

- El modelo de componentes de JSF facilita el desarrollo de bibliotecas de componentes enriquecidos
- **Bibliotecas de componentes visuales**
 - Aportan colecciones de etiquetas con componentes de funcionalidad enriquecida
 - Utilizan espacios de nombres diferentes e.g. <p:button>
 - Ejemplos:
 - [Primefaces](#) (basada en JQuery/JQueryUI)
 - [IceFaces](#)
 - [Boostfaces](#), [Butterfaces](#)
 - [RichFaces](#) (JBoss JEE Application Server)
 - [ADF Faces](#) (Oracle Application Server)
- **Utilidades**
 - Aportan principalmente clases de utilidad y “mejoras” al API JSF
 - Ejemplos
 - [OmniFaces](#), clases de utilidad
 - [PrettyFaces](#), uso de URLs Restful en JSF

47

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Conclusiones

- Jakarta Faces es el framework para la creación de la **capa de presentación** en aplicaciones web JEE
- JSF es un framework tipo *pull-based* que favorece el **prototipado rápido** de aplicaciones web con interfaz enriquecida (RIA)
- JSF está basado en un modelo de desarrollo **orientado a componentes** que trata de abstraer en cierta medida las características de HTTP
- Los **facelets** son documentos xml que permiten modelar las vistas. Éstas a su vez son objetos instanciados de forma transparente en el servidor, que se encargan de generar contenidos html e interactuar entre el usuario y la aplicación
- Los **controladores** en JSF son beans cuyas propiedades y métodos están vinculados, respectivamente a la información y eventos que generan las vistas
- Gracias al modelo de componentes de JSF, existen **bibliotecas de componentes** que simplifican la integración de tecnologías y elementos html, css y js sin necesidad de conocer sus detalles específicos de uso.

48

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025

Bibliografía

- Jakarta Faces Technology, en *Jakarta EE Tutorial*. Eclipse Foundation, 2024. Disponible en <https://jakarta.ee/learn#documentation>
- Arjan Tijms, Bauke Scholtz. *The definitive guide to Jakarta Faces in Jakarta EE 10: building Java-Based web applications*. Second Edition. Apress, 2022. Disponible en [[Recurso electrónico](#)]
- D. R. Heffelfinger, Java Server Faces, en *Java EE 8 Application Development*. Packt Publishing, 2017. Disponible en [[Recurso electrónico](#)]

Lecturas Complementarias

- *Jakarta Faces View Definition Tags*. Eclipse Foundation, 2022. Disponible en <https://jakarta.ee/specifications/faces/4.0/vdldoc/>
- *Jakarta Faces Specification. Version 4.0*. Disponible en <https://jakarta.ee/specifications/faces/4.0/>
- *JSF Tutorial (n.d.)*. Tutorialspoint. Retrieved March 2021 from <http://www.tutorialspoint.com/jsf/index.htm>